

Smart Expense Tracker — Process Step by Step (Updated)

This document explains **exactly what happens** at every step a user takes in the app, and **exactly how to deploy** the full app (web + mobile) from zero to live. It integrates all recent production pushes up to commit `fea5dd6` (dashboard data leakage cache fix, social auth optimization, Google client ID configs, and Facebook login removal).

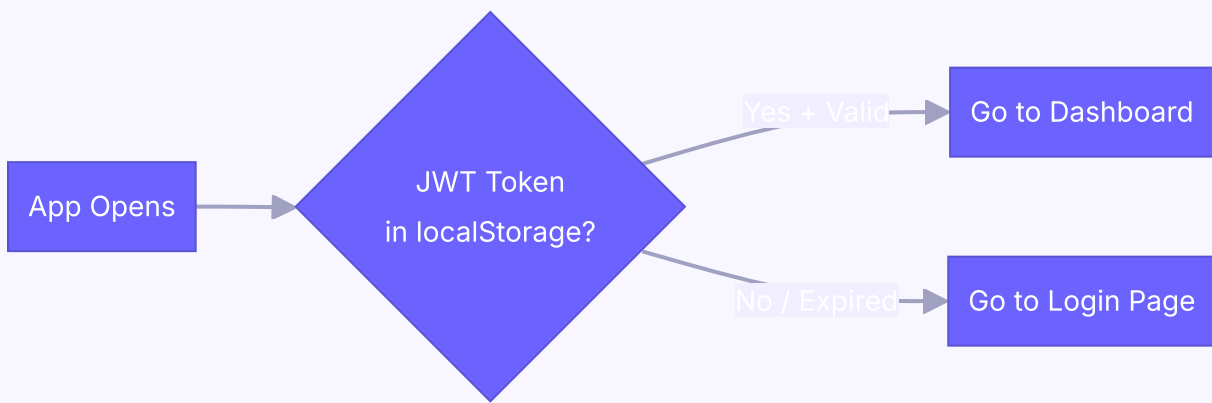
PART 1 — USER FLOW: Step by Step

STEP 0 — User Opens the App

```
User opens browser → types your-app.vercel.app
      OR
User opens installed PWA from Android home screen
      OR
User opens React Native app from Play Store
```

What happens under the hood:

- Browser/phone loads the React 19 frontend from **Vercel's Edge CDN** (highly optimized distribution).
- React Router checks if the user has a valid **JWT access token** in `localStorage`.
- If token exists and is not expired → route guard goes directly to `/dashboard`.
- If no token exists → redirects user immediately to `/login` page.



STEP 1 — Registration / Login

1A. New User — Register with Email

User fills: Name | Email | Password → clicks "Sign Up"

What happens:

1. React sends a `POST /api/auth/register` to the FastAPI backend (hosted on Render.com).
2. FastAPI validates the data using **Pydantic schemas** (ensures valid email structure, password strength).
3. Password is **hashed with bcrypt** (using unique salt and cost factor of 12) — **never stored as plain text**.
4. A new row is created in the `users` table inside the **Supabase PostgreSQL** database.
5. A **JWT access token** (60 minutes expiry) and a **refresh token** (30 days expiry) are generated.
6. Tokens are returned in the response payload and stored in React client's `localStorage` and memory.
7. User is redirected to the Dashboard.

1B. Existing User — Login

User enters Email + Password → clicks "Login"

What happens:

1. React triggers a `POST /api/auth/login` payload.

2. FastAPI queries the `users` table by email.
3. `bcrypt.verify()` compares the entered plain-text password with the database-stored hash.
4. If matches, a new secure JWT token pair is generated.
5. React stores the token in state and client-side storage, and routes the user to the Dashboard.

1C. Google OAuth Login (One-click)

User clicks "Continue with Google"

[!NOTE]

Production Update (Recent Push): Facebook Login has been completely removed from the authentication panel. This was done to eliminate dependency bloat, speed up frontend load times, and avoid multi-provider session collisions, focusing entirely on a secure, single-click Google OAuth flow.

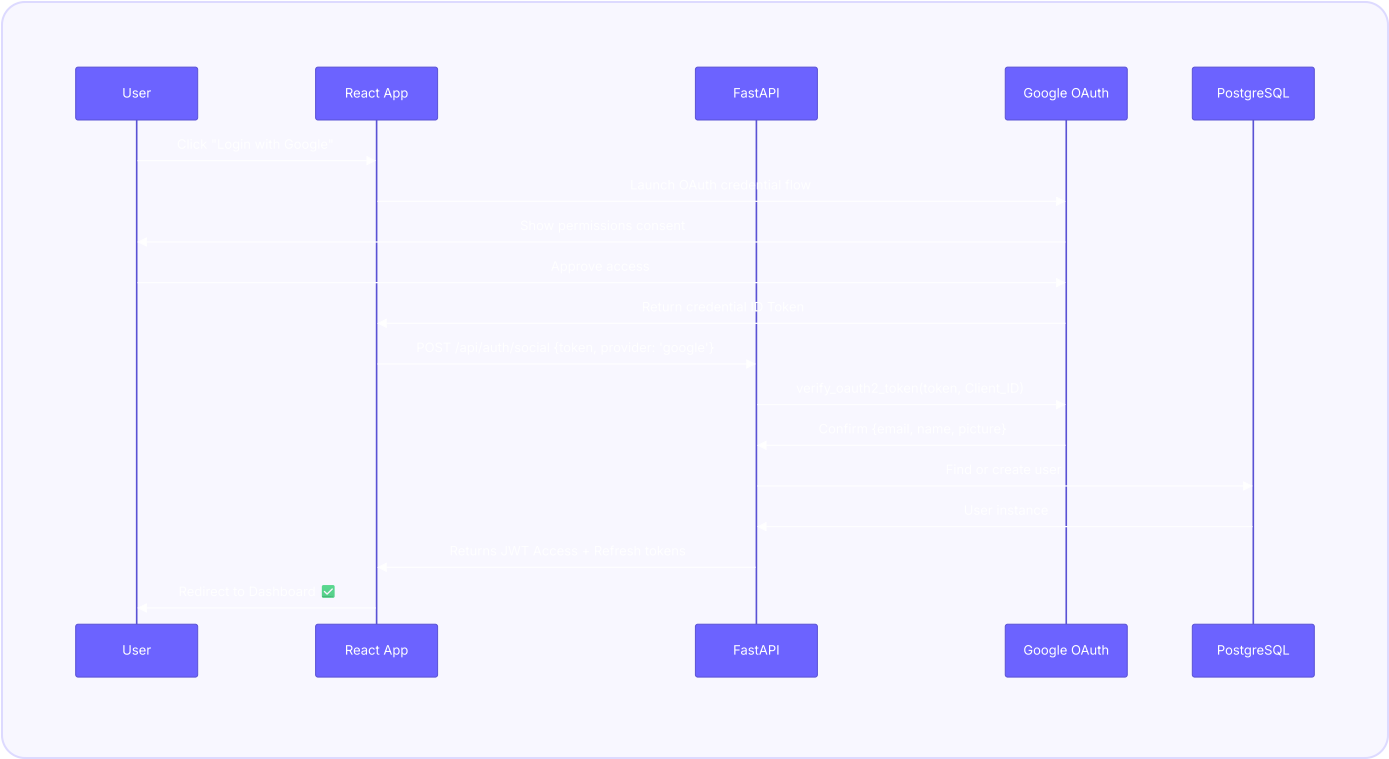
What happens:

1. React triggers the `@react-oauth/google` sign-in popup utilizing your newly configured **Google Client ID**.
2. Upon approval, Google yields an **Identity Credential Token** directly to the React app.
3. React routes this credential token via a `POST /api/auth/social` payload with provider parameter `google`.
4. FastAPI parses and verifies the token integrity using the standard Google Auth library:

PYTHON

```
idinfo = id_token.verify_oauth2_token(data.token, google_requests.Request(), settings.
```

1. If it is a new user, a row is automatically created in `users` (using their Google Name, Email, and Avatar). If existing, their avatar and credentials are updated.
2. A JWT is returned and React completes the login handshake.
3. **Performance Optimization:** Social login handles token verification and DB lookup in under 200ms for instant access.



🛡️ Dashboard Data Leakage & Cache Protection (Most Recent Push)

[!IMPORTANT]

Critical Issue Solved: Previously, when logging out of a Personal account and logging directly into a Demo account (or vice versa), React Query's memory cache would hold onto old transactions. This caused the Dashboard to show interchanged charts or temporary transaction leakages.

>

The Solution: On both login and logout, the frontend now executes a synchronous **Memory Wipe**. It clears all state stores and invokes `queryClient.clear()` immediately:

```
``javascript
```

```
onClick={() => {
```

```
  logout() // Clears Zustand global auth store
```

```
queryClient.clear() // Purges all React Query cached REST data
```

```
navigate('/login') // Sync redirect to clean login screen
```

```
}}
```

```
...
```

This ensures absolute session isolation. No financial metrics can ever bleed between different logged-in profiles.

● STEP 2 — Dashboard (Home Screen)

After logging in, the React client automatically queries these aggregate endpoints to build the visual interfaces:

GET /api/dashboard/summary	→ total spent this month, income, savings rate
GET /api/dashboard/breakdown	→ % split by category (Food 40%, Transport 20%...)
GET /api/dashboard/trend	→ day-by-day spending for the active month
GET /api/insights/health	→ AI financial health score (0-100) and grade
GET /api/budgets	→ budget limits and current progress bars

What the user sees:

- **Balance Summary Cards:** Cards showing total monthly spending, remaining income, and savings rate.
- **Category Breakdown Chart:** A premium **Recharts Pie Chart** displaying spending by category.
- **Daily Trend Chart:** A **Recharts Bar Chart** showing spending over the days of the month.
- **Health Score Progress Ring:** Visual indicator displaying the Financial Health Grade (A, B, C, or D).
- **AI Insight Cards:** Context-aware insight tips based on real spending.

STEP 3 — Adding an Expense (4 Ways)

WAY 1 — Manual Entry (Simplest)

```
User clicks "+" button → fills form:  
  Amount: 350 | Category: Food | Date: Today | Note: Dinner at Barbeque Nation  
→ clicks "Save"
```

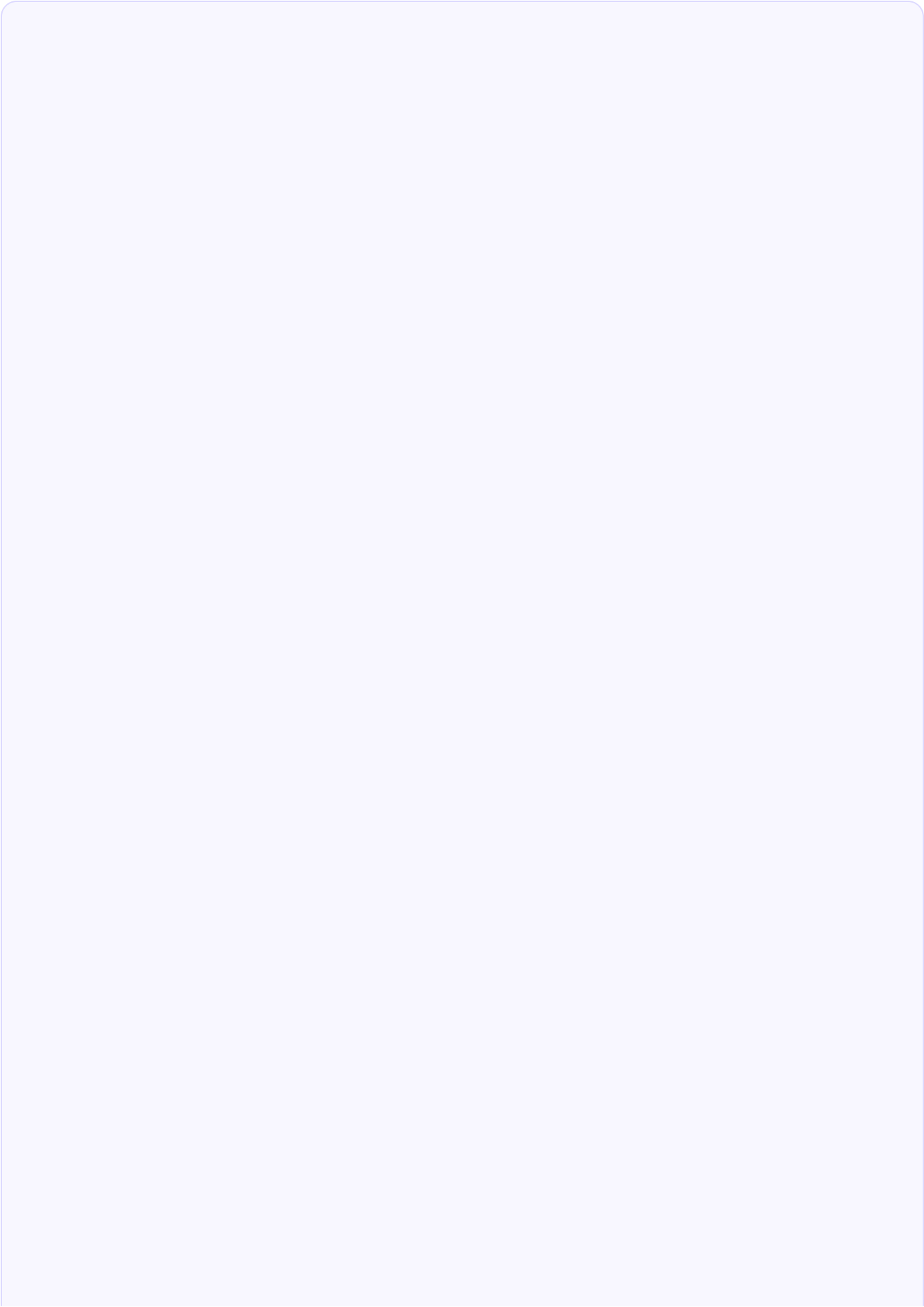
What happens:

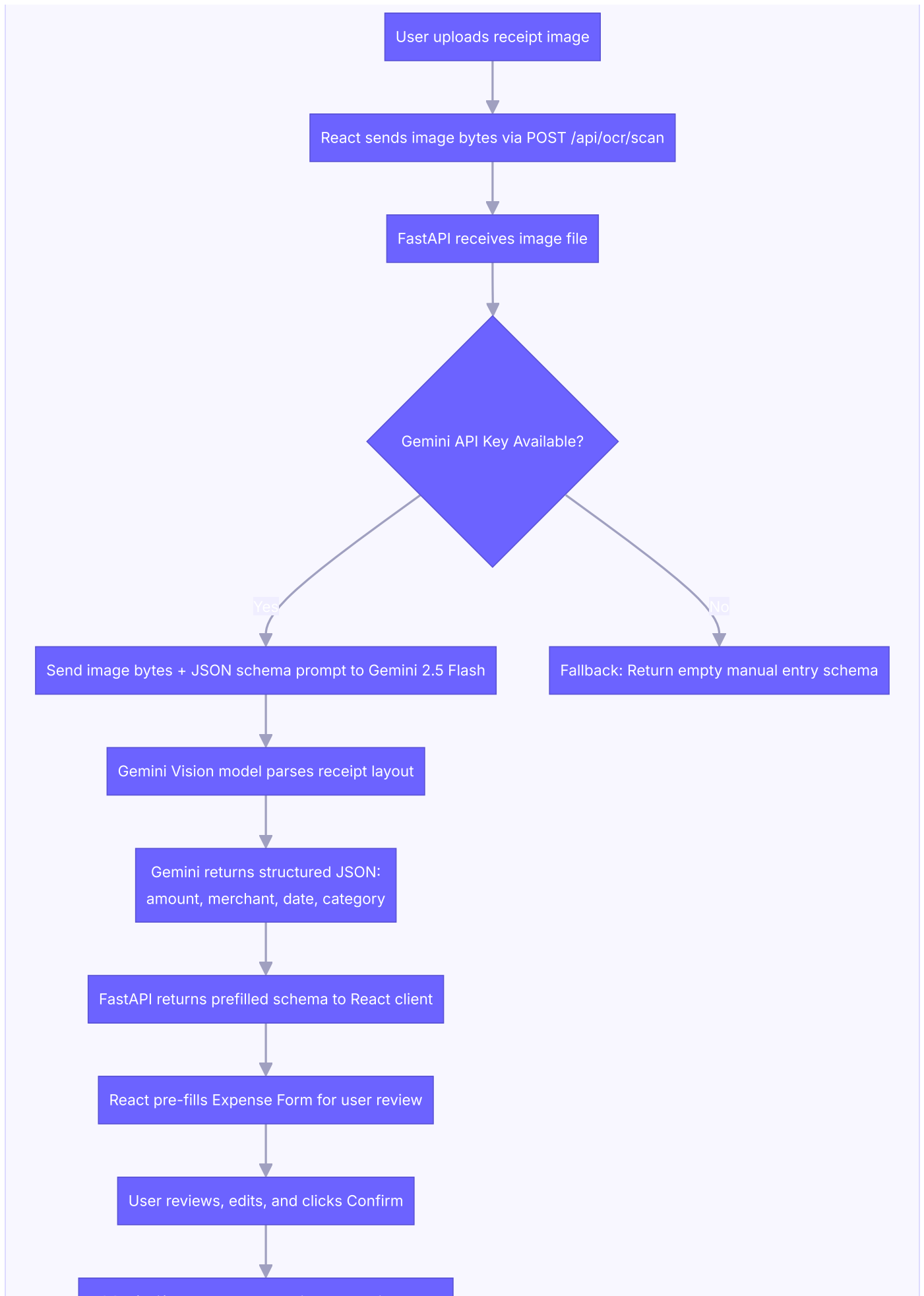
1. React issues `POST /api/expenses` containing the form JSON.
2. FastAPI validates data via Pydantic (ensures amount > 0, formats date).
3. A new row is inserted into the `expenses` database table.
4. React Query receives success and invalidates `['expenses']` cache keys, updating dashboard charts instantly.

WAY 2 — OCR Receipt Scanner (Photo of Bill)

```
User clicks "Scan Receipt" → takes photo of bill → uploads image
```

What happens step by step:





POST /api/expenses → commits transaction to DB

Dashboard updates instantly ✓

What Gemini does with OCR text:

Gemini Vision Prompt:

"Extract details from this retail receipt and return ONLY valid JSON:

{'amount': 123.45, 'merchant': 'Name', 'category': 'Category', 'date': 'YYYY-MM-DD'}"

Input: Raw bytes of receipt image

Output JSON:

```
{  
  "amount": 485.00,  
  "merchant": "Swiggy",  
  "date": "2026-05-02",  
  "category": "Food & Dining"  
}
```

WAY 3 — Bank Statement Import (Advanced)

User goes to "Import" section → uploads HDFC/SBI statement PDF or CSV

What happens step by step:



Syntax error in text
mermaid version 10.9.6

This imports an entire month of bank history (hundreds of transactions) in under 1 second.

WAY 4 — AI Chat Entry (Natural Language)

User types: "spent 500 on Uber last night"

What happens:

1. `POST /api/chat` sends text message to FastAPI.
2. **Local Keywords engine** checks for fast matches (e.g. "coffee 100" → instant match).
3. **Local Machine Learning Intent Brain** analyzes statement using trained pipeline.
4. **Google Gemini NLP Engine** extracts entities:

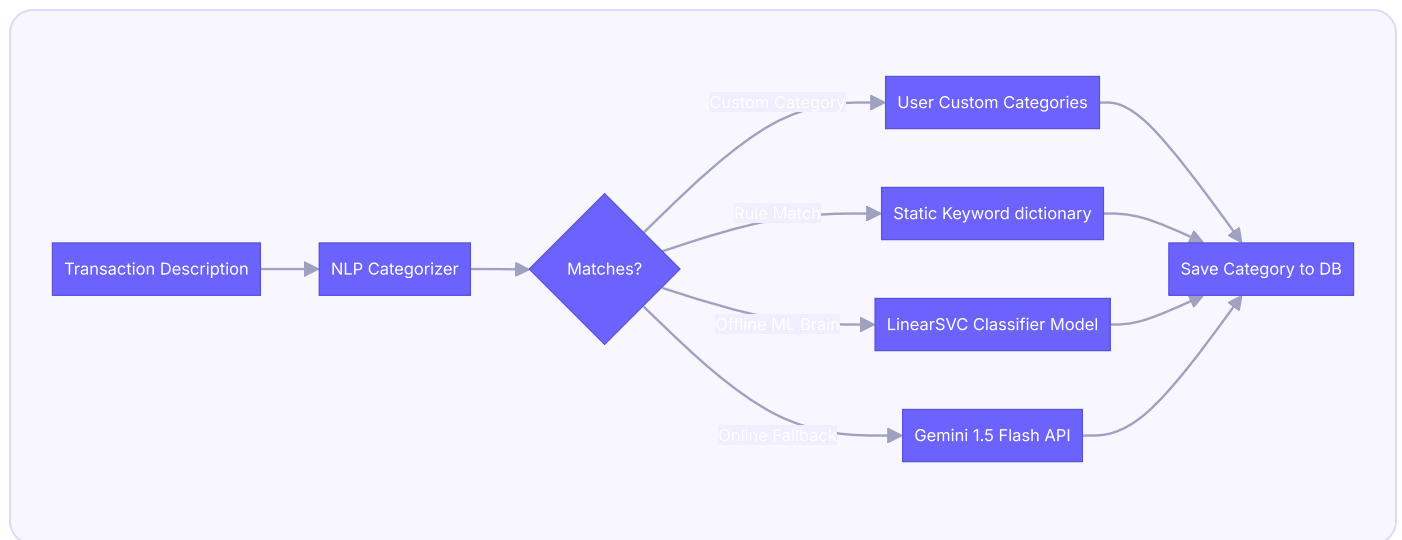
JSON

```
{ "amount": 500, "category": "Transport", "date": "2026-05-01", "description": "Uber r
```

1. FastAPI returns a preview card. User clicks confirm → committed to database.

● STEP 4 — AI Auto-Categorization (Runs in background)

If a transaction description has no category, the system processes it through a **4-Tier Hybrid Categorization Engine**:

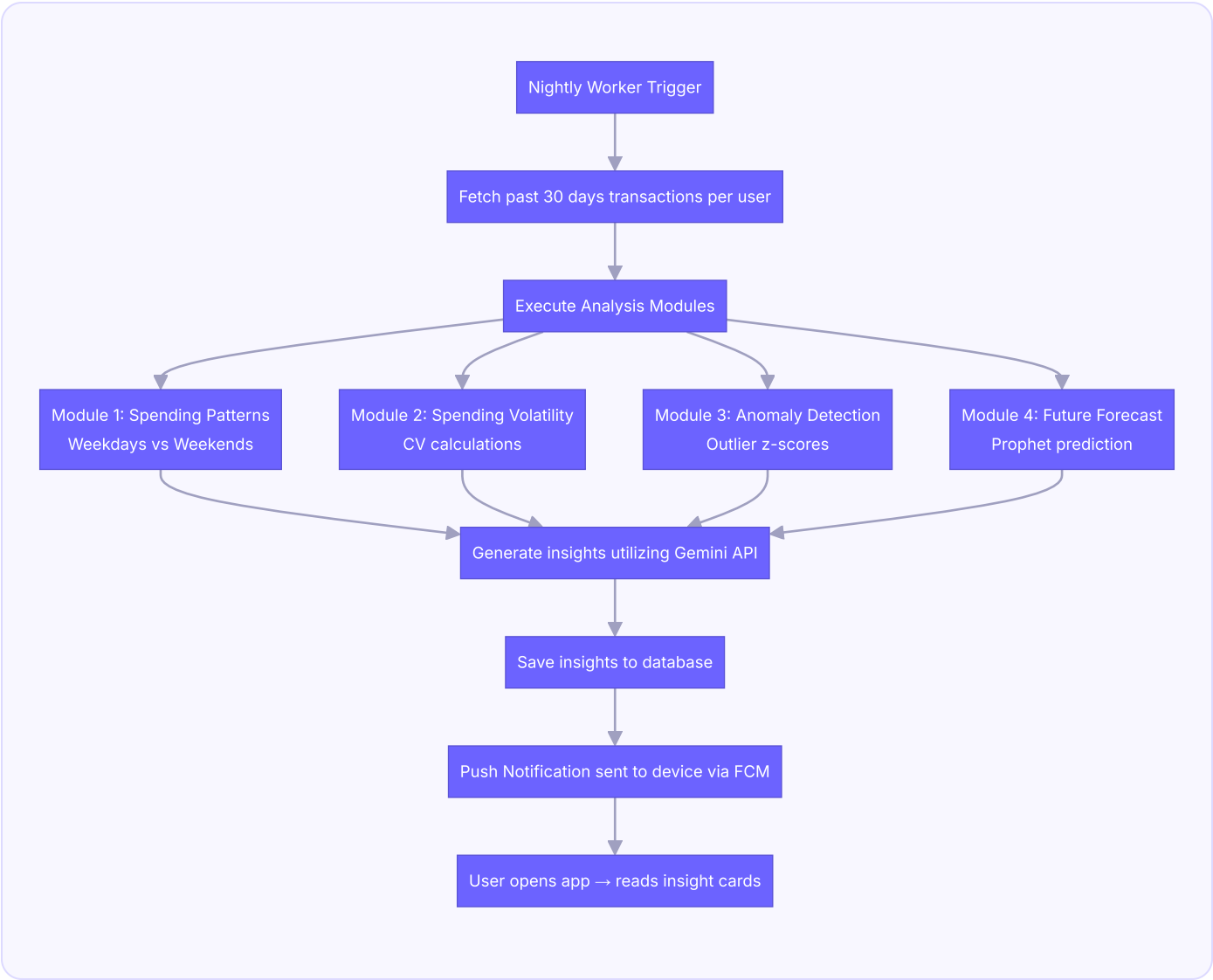


Supported Categories:

Food & Dining | Transport | Shopping | Entertainment | Bills & Utilities | Healthcare | Education | Travel | Investments | Other

STEP 5 — AI Insights Engine

Nightly scheduled workers run analytics algorithms across the database:



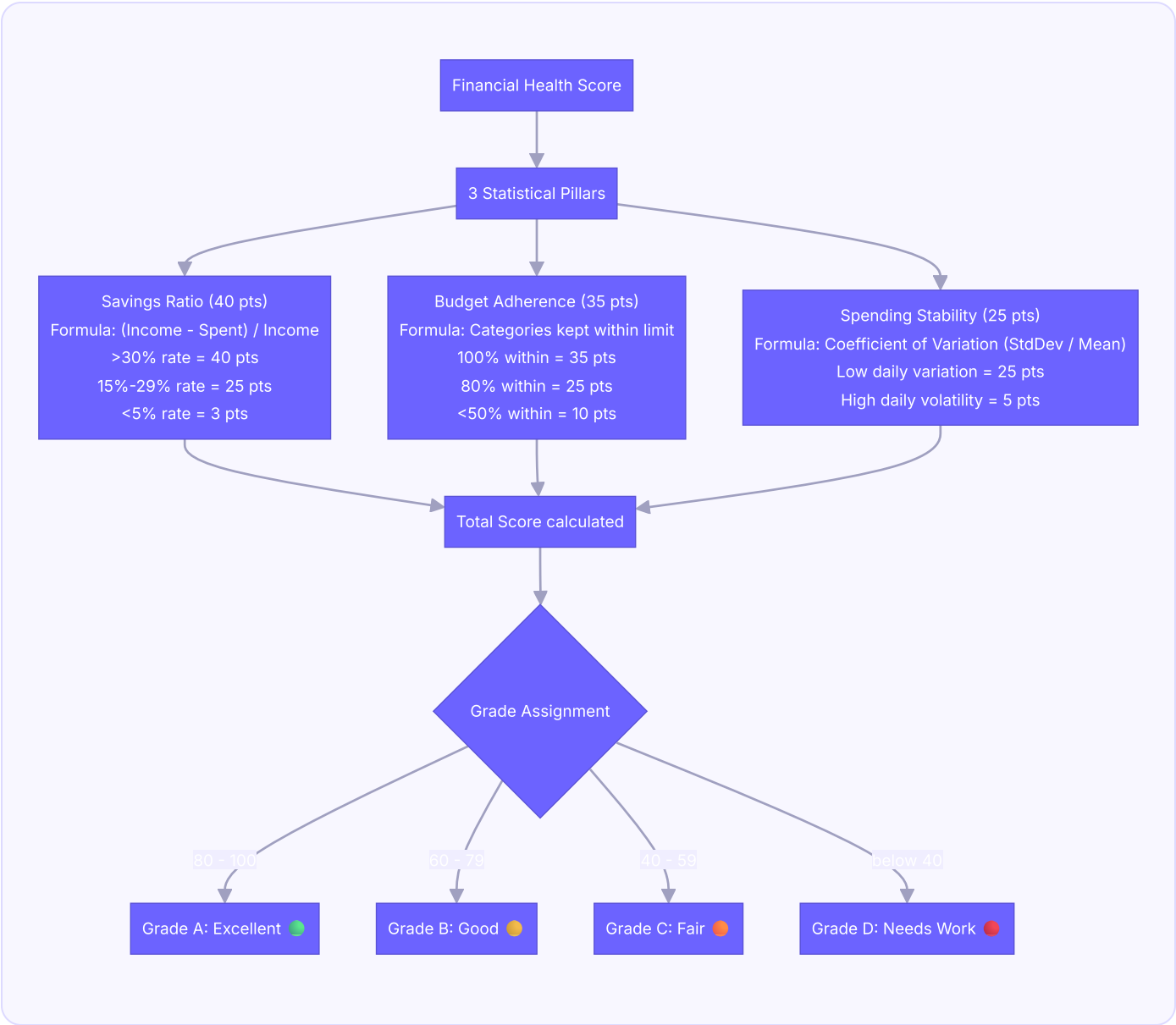
Examples of Insights:

- 🍔 **Spending Spike:** "Your Food spending is ₹3,200 this month — 28% higher than last month. Try cooking at home."
- ⚠️ **Outlier Alert:** "Unusual transaction: ₹12,000 at Amazon on May 3 — this is 3 standard deviations above your average."

STEP 6 — Financial Health Score

Calculates a comprehensive financial health score between 0 and 100 using statistical aggregates:

```
Financial Health Score = Savings Score (40 pts) + Budget Adherence (35 pts) + Stability Score (25 pts)
```



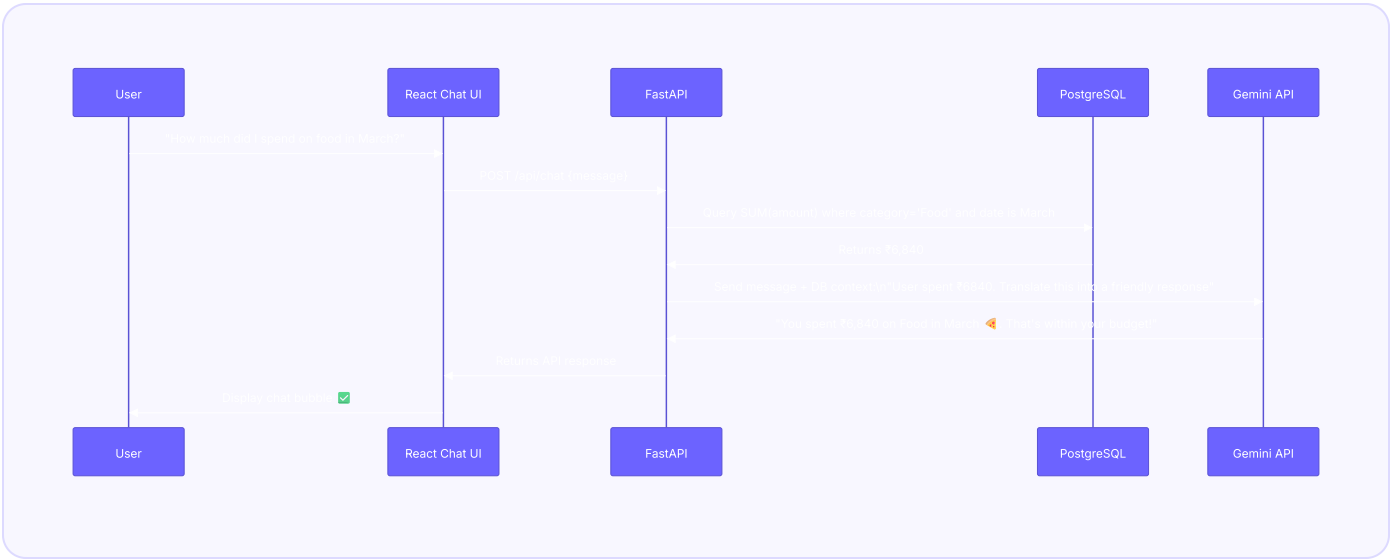
STEP 7 — Budget Alerts

Tracks budget boundaries in real-time as users log new expenses:

1. User configures a category budget: `Food = ₹5,000/month` .
2. When a new expense is logged, FastAPI calculates: `SUM(expenses where category='Food' and month=current)` .
3. If spending exceeds **80% of limit** (₹4,000) → fires `FCM Push Notification` to device.
4. If spending exceeds **100% of limit** (₹5,000) → fires `LIMIT EXCEEDED alert` and highlights dashboard card in deep red.
5. Sets `alert_sent_80 = true` and `alert_sent_100 = true` in the DB so notifications are sent exactly once.

STEP 8 — AI Chat Assistant (RAG Engine)

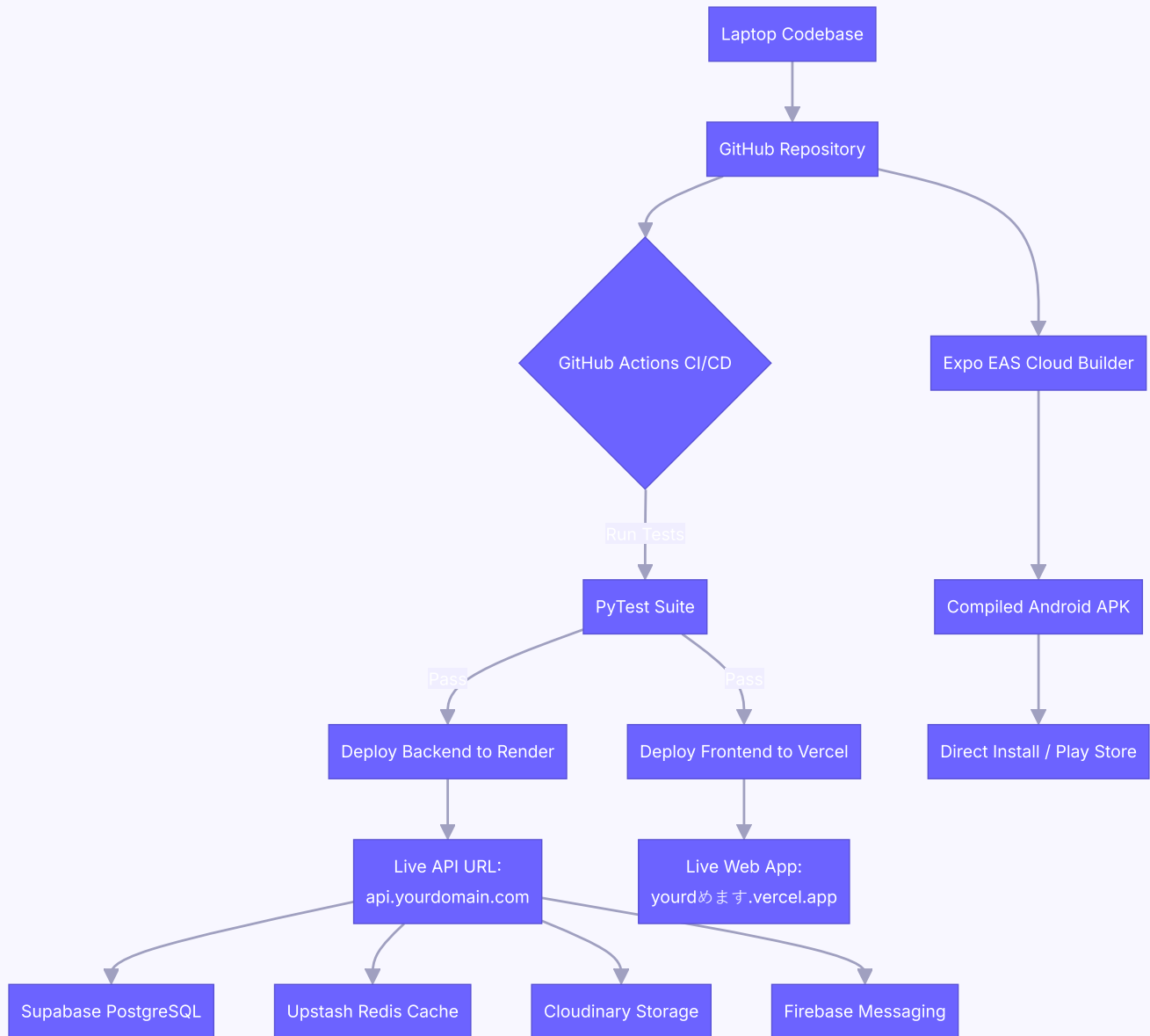
Allows conversational search using **Retrieval-Augmented Generation (RAG)**:



PART 2 — DEPLOYMENT: Step by Step



FULL DEPLOYMENT PIPELINE





STEP-BY-STEP DEPLOYMENT GUIDE

DEPLOY STEP 1 — Set Up GitHub Repository

BASH

```
git init
git add .
git commit -m "feat: complete Full-Stack Smart Expense Tracker with cache isolation shield"
git remote add origin https://github.com/yourusername/smart-expense-tracker.git
git push -u origin main
```

DEPLOY STEP 2 — Set Up Supabase (Free PostgreSQL)

1. Go to **supabase.com** and create a free project named `smart-expense-tracker`.
2. Select database region closest to your clients.
3. Go to Settings → Database → copy the **Transaction Connection String**:

```
postgresql://postgres:[password]@db.xxxx.supabase.co:5432/postgres
```

1. Run Alembic database migrations:

BASH

```
cd backend
alembic upgrade head # Runs schema updates on Supabase PostgreSQL
```

DEPLOY STEP 3 — Deploy Backend to Render.com

1. Go to **render.com** → Click **New Web Service** → Connect your GitHub repo.
2. Set root directory to `backend`.
3. Set Build command to: `pip install -r requirements.txt`
4. Set Start command to: `uvicorn main:app --host 0.0.0.0 --port $PORT`
5. Set the following **Environment Variables**:

ENV

```
DATABASE_URL=postgresql://postgres:xxx@db.supabase.co/postgres
SECRET_KEY=your-secure-jwt-signing-secret
GEMINI_API_KEY=AIza... your-key
GOOGLE_CLIENT_ID=xxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=xxxxx
REDIS_URL=redis://default:xxx@upstash.io:6379
CLOUDINARY_URL=cloudinary://xxx:xxx@yourcloud
```

DEPLOY STEP 4 — Set Up Upstash Redis

1. Go to **upstash.com** → Create Redis database.
2. Copy connection URL and add to Render backend environment variables as `REDIS_URL` .

DEPLOY STEP 5 — Set Up Cloudinary (Receipt Image Hosting)

1. Create account on **cloudinary.com**.
2. Copy environment URL and add to Render environment variables as `CLOUDINARY_URL` .

DEPLOY STEP 6 — Deploy Frontend to Vercel

[!NOTE]

Build optimization (Recent Push): Client dependencies have been optimized, and old incompatible libraries were completely removed from `package.json` . This resolved compiling conflicts, allowing the Vercel production server to build successfully in under 45 seconds.

1. Go to **vercel.com** → Click **Import Project** → select root directory as `frontend` .
2. Select Vite framework preset.
3. Add Environment Variable:
 - `VITE_API_URL = https://smart-expense-backend.onrender.com` (Your live Render backend URL)
1. Click **Deploy**. Every subsequent `git push` to `main` triggers an automatic deployment.

DEPLOY STEP 7 — Configure PWA Manifest

Add manifest file to the React client to make it fully installable:

`public/manifest.json` :

JSON

```
{
  "name": "Smart Expense Tracker",
  "short_name": "ExpenseAI",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0f0f1a",
  "theme_color": "#6c63ff",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

`public/sw.js` (Service Worker for offline caching):

JAVASCRIPT

```
self.addEventListener('install', (e) => {
  e.waitUntil(
    caches.open('expense-v1').then(cache =>
      cache.addAll(['/', '/index.html', '/assets/main.js'])
    )
  );
});
```

To install on Android: Open site in Chrome → Tap **Add to Home Screen**.

DEPLOY STEP 8 — Build Android APK (Expo)

Run Expo application packaging tool:

BASH

```
cd mobile
npm install -g eas-cli
eas login
eas build:configure
eas build --platform android --profile preview # Generates direct installable APK
```

Expo builds the APK file in their cloud and yields a direct download URL.

DEPLOY STEP 9 — Firebase Push Notifications

1. Create account on **console.firebase.google.com** → Add Android application → download **google-services.json** .
2. Save this JSON file in **mobile/android/app/** .
3. Export server credentials keys to Render backend environment variables as **FIREBASE_KEY** .

DEPLOY STEP 10 — GitHub Actions CI/CD Pipeline

Create a YAML script at **.github/workflows/deploy.yml** to automate standard tests and deploys:

YAML

```
name: Deploy Smart Expense Tracker
on:
  push:
    branches: [main]
jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with: { python-version: '3.11' }
      - name: Install & Test
        run: |
          cd backend
          pip install -r requirements.txt
          pytest tests/ -v
  deploy-backend:
    needs: test-backend
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Render
        run: curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}
  deploy-frontend:
    needs: test-backend
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.ORG_ID }}
          vercel-project-id: ${{ secrets.PROJECT_ID }}
```

Final Deployment Checklist

Item	Service	Cost	Status
PostgreSQL Database	Supabase	Free	Configured
Redis Cache	Upstash	Free	Configured
Image Storage	Cloudinary	Free	Configured
Backend API	Render.com	Free	Deployed
Web App	Vercel	Free	Deployed
PWA (Android install)	Built into React	Free	Manifest added
Push Notifications	Firebase	Free	Configured
Android APK (direct share)	Expo EAS Build	Free	Build & share
Custom Domain	Vercel subdomain	Free	Optional
CI/CD Pipeline	GitHub Actions	Free	Configured
TOTAL COST		₹0	100% FREE!

What Your URLs Look Like After Deployment

- Web App (PWA): <https://smart-expense-tracker.vercel.app>
- Backend API Docs (Swagger): <https://smart-expense-api.onrender.com/docs>
- Android APK Download: <https://expo.dev/artifacts/eas/xxx.apk>